Q1 — Deep Speed Question

If NumPy and Python lists both store numbers, why does NumPy still need less CPU work per operation?

Explain in terms of:

- Type checking
- Memory access
- Interpreter involvement

1. List can store elements of any data types that are of different memory sizes, this habit compels CPU to do type checking that is a single reason of why lists are slow. And this doesn't happen in NumPy.
   NumPy store elements of same data type and size, no type checking, and CPU can perform operations directly.

2. List store references to objects and objects are scattered.
   NumPy stored actual values which are in continuous memory block, it helps in better CPU cache usage and memory access that does not happen in lists.

3. In lists to perform any operation, you must loop the list and then the desired operation will be performed that is the result of slow speed.
   In NumPy array executed in optimized C code internally, so no loop and interpreter involved, uses SIMD.

Q2 — Data Buffer Logic

Imagine an `ndarray` without metadata.

What problems would occur if:

- shape did not exist?
- dtype did not exist?
- strides did not exist?

1. If shape did not exist, then you don't know how to interpret that data as you are unaware of the shape.
2. If dtype did not occur then you can't know the memory usage of array, speed and precision.

3. If strides did not exist, then how can you know that how many memory blocks to jump to move to next element in each dimension.

Q3 — dtype Intelligence

If I create:

```
np.array([1, 2, 3.5])
```

Why does NumPy convert everything to float automatically?

What rule is it following?

What would happen if it did NOT do that?

1. NumPy convert everything to float automatic because it must keep every element of the same data type and this process is called **upcasting**.
2. It follows the rule named upcasting.
3. If it did not do that then different elements would be of different data types and size like lists, and CPU must perform type checking as well.

Q4 — Memory Mathematics

You create:

- 1000 elements
- dtype = float64

Without calculator:
How much memory (in bytes) does this consume?

Now answer:
Why would choosing float32 instead matter in large datasets?

1. nbytes = size * itemsize
   nbytes = 1000 * 8
   nbytes = 8000

2. float32 uses less memory as compared to float64, more efficient and faster.

Q5 — Strides Understanding

In simple words:

If strides were wrong or corrupted, what kind of errors would you expect when accessing array elements?

1. As strides tell you how many memory blocks to jump to move to next element in each dimension. If you don't know the strides, then how you can know that how many memory blocks I need to jump to move to next row or column.

Q6 — View vs Copy Mastery

Why did NumPy designers choose slicing to return a view instead of a copy?

Think in terms of:

- Performance
- Large datasets
- Scientific computing

You might access wrong memory location.

Indexing would return wrong values.

Q7 — Advanced Thinking

If two arrays share the same data buffer but have different shapes,
are they different arrays or the same array?

They are the different ndarray objects because the other is probably a view of first array that shares the same memory buffer.